

NexusSim — Explained

A city-scale traffic simulation platform, extended into an integrated
Reinforcement Learning · Computer Vision · NLP · Soft Computing system

This document explains what NexusSim is today, why it is a natural host for four academic subjects, and how those subjects are designed to interoperate as one system rather than four separate demos bolted onto a shared repository.

Companion document: IMPLEMENTATION_PLAN.md (phase-by-phase build plan, Phases 0–18) — this document is the narrative explanation; the plan is the execution checklist.

1. What NexusSim Is

NexusSim is a high-fidelity, real-city traffic simulator built to let planners and researchers test policy changes — a new bus lane, a congestion-pricing zone, a different signal-timing strategy — before committing to them in the real world. It simulates thousands of individually-modeled vehicles and pedestrians moving through a real city's road network, and treats **equity** (are the benefits distributed fairly across neighborhoods?) as a first-class output alongside raw efficiency.

The project is built in three cooperating subsystems, each with a hard interface boundary so it can be developed independently:

- **C++ simulation engine** — loads a real city's road graph, spawns and moves agents (cars, buses, auto-rickshaws, two-wheelers, pedestrians) tick by tick, and streams live state to any listener over WebSocket.
- **Python data pipeline** — turns raw OpenStreetMap data and real trip-demand data (currently Chicago's Transportation Network Provider data) into the graph and origin-destination files the engine consumes.
- **React dashboard** — renders the live simulation on a map, shows efficiency and equity metrics in real time, and is the surface through which a human interacts with the system.

1.1 Engine internals

Each simulated vehicle follows the **Intelligent Driver Model (IDM)** for car-following behavior, with parameters tuned per vehicle type. Route choice uses **A* pathfinding**, with a stochastic variant that adds random noise to edge costs so not every driver takes the mathematically optimal route — a closer approximation of real human route diversity. A **quadtree spatial index** keeps neighbor-finding (for collision avoidance and lane changes) fast even at 20,000+ concurrent agents. Intersections currently run on **Webster's formula** — a classical fixed-cycle signal-timing method computed once from initial traffic volumes.

1.2 What's already validated

The pipeline has already ingested real Chicago trip data and calibrated the engine's car-following parameters against it. A validation script runs the engine headless and compares simulated corridor journey times against the real-world ground truth, checking that simulated times land within 25% of observed values for at least 75% of corridors — this is the project's current "is the simulation realistic" checkpoint.

1.3 Where the four subjects fit

None of Reinforcement Learning, Computer Vision, NLP, or Soft Computing exist in the codebase yet — an ml/ directory is scaffolded but empty. Crucially, though, the existing architecture already has natural attachment points for each:

Subject	Natural attachment point in the existing system
Reinforcement Learning	Replace the fixed-cycle signal controller with a learned policy — the engine already ticks a controller per inte

Subject	Natural attachment point in the existing system
Soft Computing	The validation script's manual parameter search is already an optimization problem waiting for a genetic algorithm
Computer Vision	No existing hook — new addition, but pairs naturally with the existing per-city, per-zone congestion metrics to
NLP	No existing hook — new addition, but the engine already streams rich live metrics that are a natural target for

2. Vision: One System, Not Four Demos

The goal is not to attach four independent proof-of-concepts to the same repository. It is for each subsystem to visibly affect the others during a live demo. Concretely:

- An **NLP incident report** ('accident on Michigan Ave, lane closure') changes the live simulation's road network, which the **RL signal policy** reacts to automatically (queues shift, so its observations shift), and the change is visible on the dashboard.
- **CV-derived real-world congestion data** feeds into the **Soft Computing** genetic algorithm as an additional calibration signal, tying real-world sensing to simulation realism.
- The **RL, fuzzy-logic**, and classical **Webster's** signal strategies all run in the same engine binary and can be swapped live from the dashboard for a direct side-by-side comparison.
- A **synthetic virtual camera** (Computer Vision) watches the live simulation itself and performs real detection/counting on the rendered frame — a working CV pipeline with no external data dependency, safe to demo in isolation.

One correction worth flagging up front: the original idea of pulling live traffic conditions from Google Maps does not hold up as Computer Vision — the Static Maps API does not expose a scriptable live-traffic layer, and scripting the interactive map view would violate Google's Terms of Service. The plan substitutes **Mapbox Traffic Tiles** or the **TomTom Traffic Flow API**, both of which expose the same kind of color-coded congestion imagery through a documented, ToS-compliant API.

3. Integrated Architecture

The extension is wired through the engine's **existing WebSocket JSON protocol** and the **existing cities.yaml config-driven pattern**, rather than inventing new plumbing. Two kinds of new components are added:

- **Engine-native additions** (real-time, C++): a pluggable signal-policy interface (Webster's / Fuzzy / RL-via-ONNX, hot-swappable with no restart), and an "apply incident" hook that temporarily changes road-edge speed/capacity.
- **Python sidecar services** (non-real-time): a chat/incident-parsing NLP service and a virtual-camera CV service, each a lightweight client of the engine's existing WebSocket stream, exposing their own REST/WS endpoints for the dashboard to call.
- **Offline/batch pipeline stages**: real-world CV congestion classification and the genetic-algorithm calibration optimizer, both Python-only, feeding static JSON reports the dashboard reads once and RL training reads during offline training.

3.1 Why RL training happens offline, in Python

The simulation engine ticks every 0.1 simulated seconds in real time. A live Python round-trip per tick would blow that budget many times over. So training happens offline: a Python environment wraps the engine's WebSocket interface as a Gym-style environment, trains a policy (PyTorch, MAPPO-style — one agent per intersection, trained against a shared network-wide reward), and the trained policy is exported

to **ONNX** format. At runtime, the C++ engine loads that ONNX model directly and runs inference in a background thread — no Python process is ever in the simulation's hot path.

3.2 Data flow

From	To	What flows, and why
NLP incident parser	Engine (WebSocket)	Structured incident spec (affected roads, severity, duration) — mutates live simulation
Engine	Dashboard, NLP chat service, CV real-time processing	Average congestion and metrics — the shared real-time state every consumer needs
CV real-world congestion pipeline	Soft Computing GA, RL reward shaping	Real-world congestion levels — an additional calibration/training signal, often used for RL
Soft Computing GA	Engine startup parameters	Tuned calibration parameters (speed factor, route spread, chaos, demand) — used to initialize the engine
RL training	Engine (ONNX policy file)	A trained signal-control policy the engine loads and runs at native speed.

4. Reinforcement Learning — Adaptive Signal Control

What it replaces: the current fixed-cycle Webster's-formula signal timing, which computes green-light durations once at startup from initial traffic volumes and never adapts to what's actually happening at the intersection.

Design: each signalized intersection gets its own independent RL agent — this is "multi-agent" in the sense that matters for the coursework, without the added complexity of agents exchanging messages with each other. All agents are trained together against one shared, network-wide reward, so an intersection's policy is shaped not just by its own queue but by how its decisions affect the whole city's average wait time and equity.

Element	Design choice
Decision timing	At phase boundaries (not every 0.1s tick) — extend the current green phase, or switch to the next one.
Observation	Queue length per approach, current phase, time in phase, time of day, and neighboring intersections' congestion
Reward	This intersection's own average wait time, plus a weighted term for the whole network's average wait time and equity
Training	Offline, in Python, via a Gym-style environment wrapping the engine's WebSocket interface.
Runtime	Exported to ONNX; inference runs natively in C++, target latency under 8 milliseconds per batched decision.

Baseline comparison: because Webster's formula and the RL policy both implement the same underlying interface, the dashboard can switch an intersection — or the whole city — between them live, and show the wait-time and equity difference immediately.

5. Computer Vision — Two Complementary Pipelines

5.1 Real-world congestion classification

Traffic-flow tile imagery (color-coded by congestion level, from Mapbox or TomTom) is fetched per city and classified two ways, deliberately built as a comparison: a **classical** approach (convert to HSV color space, threshold road-colored pixels, bucket into a congestion level) that needs no training data, and a **learned** approach (a small CNN, or a fine-tuned ResNet-18) trained on a hand-labeled sample. The output feeds into the Soft Computing calibration step as a real-world sanity check on the simulation's emergent congestion patterns.

5.2 Synthetic virtual camera

A second, independent CV pipeline renders a top-down view directly from the live simulation — roads from the map data, agents as colored shapes — and then runs genuine OpenCV detection (contour/blob detection with color segmentation) on that rendered frame to detect and count vehicles, exactly as if watching a real traffic camera. Because it only reads the same live stream the dashboard already reads, it carries no risk to the simulation's control path and can be built and demoed independently of everything else.

6. NLP — Two Complementary Interfaces

6.1 Live metrics chat

A chat panel answers questions like "which zone has the worst wait time right now" against the simulation's actual current state, not a static snapshot. The core is a small, reliable rule-based intent classifier (a fixed set of question types matched by keyword/pattern), with an optional large-language-model layer added on top for a direct "classical NLP vs. LLM" comparison — the system still works with no API key configured, falling back to the rule-based path.

6.2 Incident reports

A free-text incident report — "accident on Michigan Ave, lane closure" — is parsed into a structured specification: which road segments are affected (matched fuzzily against the city's real street names), what kind of incident it is, and how long it lasts. That specification is sent to the running simulation, which temporarily reduces speed/capacity on the affected roads. This is the clearest cross-subsystem demo in the whole project: submitting one incident report visibly changes routing on the map, is picked up automatically by the RL signal policy's observations, and shows up in the dashboard's live incident list — all from one text box.

7. Soft Computing — Optimization Without Gradients

7.1 Genetic algorithm calibration

The project already has a calibration problem: several simulation parameters (driver speed factor, route-choice randomness, driving-chaos coefficient, demand scaling) are currently tuned by hand against a real-world accuracy checkpoint (simulated vs. observed corridor journey times). A genetic algorithm replaces that manual search: each candidate parameter set is a chromosome, its fitness is the accuracy score from running the existing validation script, and the population evolves through selection, crossover, and mutation across generations — each candidate evaluated as an independent, parallel simulation run.

7.2 Fuzzy-logic signal controller

A second, non-learned alternative to both Webster's formula and the RL policy: a fuzzy-logic controller that reasons about "long" or "short" queues and wait times using fuzzy membership functions and a small human-readable rule base (e.g. "if the queue is long and the wait is long, extend the green phase significantly"), producing a concrete green-time extension through centroid defuzzification. It slots into the same pluggable signal-policy interface as Webster's and RL, so all three can be compared directly.

8. Team Roadmap

For a team of three to four people, the work breaks into a short shared foundation phase, then four largely-parallel subsystem tracks, then an integration phase. Full task-by-task detail (branch names, checkpoints) lives in **IMPLEMENTATION_PLAN.md**, Phases 12–18. Summary:

Stage	Who	Depends on
Shared plumbing	Whole team together	Nothing — do first
Reinforcement Learning	1 person	Shared plumbing
Computer Vision (both pipelines)	1 person	Shared plumbing
NLP (both interfaces)	1 person	Shared plumbing
Soft Computing	1 person (or folded into RL owner on a 3-person team)	Shared plumbing
Dashboard integration	Rotates / integration lead	Each subsystem's output format frozen
Hardening & integrated demo	Whole team together	All subsystems working individually

Demo script: start the engine with GA-tuned parameters and a trained RL policy loaded; open the dashboard and toggle between Webster's, fuzzy, and RL signal control while watching wait-time and equity metrics change; toggle the CV congestion overlay and watch the virtual camera detect and count live traffic; ask the chat panel a question about current conditions; submit a text incident report and watch the map, the RL policy's behavior, and the metrics all react together.